

Operating System Interview Questions - Ranked by Importance

TOP 10 MOST IMPORTANT OS INTERVIEW QUESTIONS

Q1: What is a Process and Process Table?

Answer: A process is an instance of a program that is currently being executed. It includes the program code, data, and the resources required for execution. The operating system manages all active processes and keeps track of them using a process table, which stores information about each process.

Polished Answer: A process is a program in execution - it's the active entity that contains the program code, its current activity (program counter, registers), and allocated resources like memory and file descriptors. The process table is a data structure maintained by the OS kernel that tracks every active process, storing critical metadata including Process ID (PID), process state, program counter, CPU registers, memory allocation details, and scheduling information. This table enables the OS to manage, schedule, and resume processes efficiently.

TL;DR: Process = program in execution. Process Table = OS's tracking system for all active processes.

Keyword/Key mappings: Process Control Block (PCB), PID, process state, program counter, CPU scheduling, memory allocation

Q2: What are the Different States of a Process?

Answer: A process passes through different states during its execution, depending on whether it is waiting for the CPU, executing, or waiting for an event. These states help the operating system schedule and manage processes efficiently.

Polished Answer: A process transitions through five primary states during its lifecycle:

- **New:** Process is being created
 - **Ready:** Process is loaded in memory, waiting for CPU allocation
 - **Running:** CPU is actively executing the process instructions
-

- **Waiting/Blocked:** Process is waiting for an event (I/O completion, user input, signal)
- **Terminated:** Process has finished execution

State transitions occur due to scheduler decisions, I/O operations, and process termination. This state model is fundamental to understanding CPU scheduling and process management.

TL;DR: New Ready Running Waiting Terminated. States help OS manage CPU allocation.

Keyword/Key mappings: Process lifecycle, state transitions, CPU scheduler, I/O waiting, process states

Q3: What is Deadlock? What are the Necessary Conditions?

Answer: Deadlock is a situation where a set of processes are blocked as each process is holding resources and waits to acquire resources held by another process. Necessary conditions: Mutual Exclusion, Hold and Wait, No Pre-emption, Circular Wait.

Polished Answer: Deadlock occurs when two or more processes are permanently blocked, each waiting for a resource held by another process in the set. Four necessary conditions (Coffman conditions) must hold simultaneously:

1. **Mutual Exclusion:** At least one resource is non-shareable (only one process can use it at a time)
2. **Hold and Wait:** A process holds at least one resource while waiting for additional resources
3. **No Preemption:** Resources cannot be forcibly removed; they must be released voluntarily
4. **Circular Wait:** A cycle exists where P1 waits for P2's resource, P2 waits for P3's resource, and so on

Deadlock prevention breaks at least one condition, avoidance uses algorithms like Banker's Algorithm, and detection/recovery handles deadlocks after they occur.

TL;DR: Deadlock = processes stuck waiting for each other's resources. Four conditions: Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait.

Keyword/Key mappings: Coffman conditions, Banker's Algorithm, resource allocation graph, circular wait, deadlock prevention

Q4: What is the Difference Between Process and Thread?

Answer: A process is an independent program under execution with its own address space. A thread is the smallest unit of CPU scheduling that runs within a process, sharing code, data, and heap with other threads but having its own stack and registers.

Polished Answer:

Aspect	Process	Thread
Definition	Independent program under execution	Smallest unit of execution within a process
Memory	Own address space (code, data, stack, heap)	Shares process memory, own stack/registers
Communication	IPC (slower)	Shared memory (faster)
Context Switching	Heavy (full state save/restore)	Lightweight (thread-specific only)
Creation	More resource-intensive	Less resource-intensive
Fault isolation	One crash doesn't affect others	Crash affects all threads in process

Processes provide isolation; threads provide concurrency within that isolation.

TL;DR: Process = independent program with own memory. Thread = lightweight unit sharing process memory. Threads are faster to create/switch.

Keyword/Key mappings: Address space, IPC, context switching overhead, lightweight process, shared memory

Q5: What is Virtual Memory?

Answer: Virtual memory is a memory management technique that allows the operating system to use a portion of secondary storage (disk) as an extension of RAM. It gives each process the illusion of having a large, continuous memory space.

Polished Answer: Virtual memory decouples logical memory from physical memory, enabling:

- **Large address spaces:** Programs can be larger than physical RAM

- **Process isolation:** Each process has its own virtual address space
- **Memory protection:** Processes cannot access each other's memory
- **Efficient multitasking:** Only active pages reside in RAM, inactive pages on disk

Implementation uses paging (fixed-size pages mapped to frames) or segmentation. The Memory Management Unit (MMU) translates virtual addresses to physical addresses using page tables. Page faults occur when referenced pages aren't in RAM, triggering loading from disk.

TL;DR: Virtual memory = using disk as extended RAM. Enables larger programs, isolation, and protection through paging.

Keyword/Key mappings: Paging, MMU, page table, page fault, demand paging, address translation

Q6: What is Thrashing and How to Prevent It?

Answer: Thrashing is a condition in which the operating system spends more time handling page faults than executing processes. It occurs when there is insufficient physical memory, causing excessive swapping of pages between RAM and disk.

Polished Answer: Thrashing occurs when a system's page fault rate is so high that the CPU spends most time swapping pages rather than executing processes. This happens when working sets of active processes exceed available physical memory.

Signs: Extremely high disk I/O, low CPU utilization, system becomes unresponsive

Solutions:

- Reduce degree of multiprogramming (run fewer processes)
- Increase physical memory
- Implement working-set model to track required pages
- Use page fault frequency (PFF) monitoring to trigger process suspension

The working-set model ensures a process only runs if its currently-needed pages fit in memory, preventing thrashing before it occurs.

TL;DR: Thrashing = excessive paging, CPU doing more swapping than executing. Fix: reduce processes or add RAM.

Keyword/Key mappings: Page fault rate, working set, multiprogramming degree, paging, memory pressure

Q7: What is a Kernel? What are Its Types?

Answer: A kernel is the core component of an operating system that manages communication between software and hardware. It controls system resources such as CPU, memory, and I/O devices.

Polished Answer: The kernel is the privileged core of the OS, running in kernel mode with direct hardware access. It provides:

- Process scheduling and management
- Memory management
- File system management
- Device driver interfaces
- System call handling

Types of Kernels:

- **Monolithic:** All services in kernel space (Linux, Unix) - faster but larger
- **Microkernel:** Minimal kernel, services in user space (QNX, Minix) - modular but slower
- **Hybrid:** Combination of both (Windows NT, macOS XNU)
- **Exokernel:** Minimal abstraction, direct hardware access for applications

TL;DR: Kernel = OS core managing hardware and resources. Types: Monolithic, Microkernel, Hybrid.

Keyword/Key mappings: Kernel mode vs user mode, system calls, monolithic vs microkernel, hardware abstraction

Q8: What is Context Switching and Its Overhead?

Answer: Context switching is the process of saving the state of the currently running process and loading the state of another process so that the CPU can switch execution between them.

Polished Answer: Context switching is the mechanism enabling multitasking by rapidly switching CPU between processes. Steps:

1. Save current process state (PC, registers, stack pointer) to its PCB
2. Update process state (Running Ready)

3. Select next process from ready queue (scheduler decision)
4. Load its saved state from PCB
5. Update state (Ready → Running)
6. Resume execution

Overhead: Context switch is pure overhead - no useful work occurs during the switch. It costs CPU cycles for state saving/loading, cache invalidation, and TLB flushing. High context switch rates degrade performance significantly.

TL;DR: Context switching = saving one process state and loading another. Costly overhead, enables multitasking.

Keyword/Key mappings: PCB, scheduler, dispatch latency, CPU registers, cache pollution, TLB flush

Q9: What are the CPU Scheduling Algorithms?

Answer: First-Come, First-Served (FCFS), Shortest-Job-Next (SJN), Priority Scheduling, Shortest Remaining Time, Round Robin (RR), Multiple-Level Queues Scheduling.

Polished Answer:

Algorithm	Type	Description	Pros	Cons
FCFS	Non-preemptive	Execute in arrival order	Simple, fair	Convoy effect, poor avg wait
SJF/SJN	Non-preemptive	Shortest burst first	Min avg waiting time	Starvation, needs burst prediction
SRTF	Preemptive	Shortest remaining time first	Optimal avg waiting	Starvation, overhead
Priority	Both	Highest priority first	Flexible	Starvation (solved by aging)
Round Robin	Preemptive	Fixed time quantum, cyclic	Fair, responsive	High context switch if quantum small
MLFQ	Preemptive	Multiple queues, dynamic priority	Adaptive, good for mixed workloads	Complex to implement

TL;DR: FCFS (order), SJF (shortest), Priority (importance), RR (time slice), MLFQ (adaptive). Each has tradeoffs.

Keyword/Key mappings: Time quantum, preemptive vs non-preemptive, starvation, aging, convoy effect

Q10: What is the Difference Between Paging and Segmentation?

Answer: Paging divides memory into fixed-size pages and frames. Segmentation divides programs into variable-size logical segments. Paging is transparent to programmer, segmentation is visible.

Polished Answer:

Aspect	Paging	Segmentation
Division	Fixed-size pages	Variable-size segments (logical)
Visibility	Invisible to programmer	Visible to programmer
Address	Page number + offset	Segment number + offset
Fragmentation	Internal	External
Purpose	Memory management efficiency	Logical organization of programs
Table	Page table	Segment table
Speed	Generally faster	Slower due to complexity

Modern systems often combine both (segmented paging) for benefits of both approaches.

TL;DR: Paging = fixed-size, transparent, internal fragmentation. Segmentation = variable-size, logical, external fragmentation.

Keyword/Key mappings: Page frames, segment table, internal/external fragmentation, logical address, MMU translation

TOP 25 QUESTIONS (11-25)

Q11: What are the Differences Between User-Level and Kernel-Level Threads?

Answer: User-level threads are managed by thread libraries without kernel awareness. Kernel-level threads are managed directly by the OS kernel, which recognizes and schedules

them.

Polished Answer:

Aspect	User-Level Threads	Kernel-Level Threads
Management	Thread library (user space)	OS kernel
OS awareness	Invisible to kernel	Visible and scheduled by kernel
Context switch	Fast (no kernel involvement)	Slower (kernel mode transition)
Blocking	One blocking thread blocks all	One blocking thread doesn't block others
Multiprocessing	Cannot utilize multiple CPUs	Can run on multiple CPUs
Implementation	POSIX threads (Pthreads)	Linux threads, Windows threads

TL;DR: User-level: fast but limited. Kernel-level: slower but more powerful. Hybrid models exist.

Keyword/Key mappings: Thread library, blocking operations, multiprocessor utilization, kernel mode, POSIX threads

Q12: What is the Difference Between Preemptive and Non-Preemptive Scheduling?

Answer: Preemptive scheduling allows the OS to interrupt a running process.

Non-preemptive scheduling lets a process run until it finishes or voluntarily gives up the CPU.

Polished Answer:

Aspect	Preemptive	Non-Preemptive
Interruption	OS can preempt running process	Process runs until completion/block
Responsiveness	Higher (good for interactive systems)	Lower (batch processing)
Context switching	More frequent, higher overhead	Less frequent, lower overhead
Complexity	More complex	Simpler

Examples	Round Robin, SRTF, Preemptive Priority	FCFS, Non-preemptive SJF
Starvation	Possible (Priority scheduling)	Possible (SJF)

TL;DR: Preemptive = OS can interrupt. Non-preemptive = process keeps CPU until done.

Preemptive is more responsive.

Keyword/Key mappings: Time quantum, priority scheduling, context switch overhead, interactive systems, batch processing

Q13: What are Semaphores and Their Limitations?

Answer: Semaphores are synchronization tools for coordinating access to shared resources among multiple processes or threads. They use wait() and signal() atomic operations.

Polished Answer: Semaphores are integer variables with atomic wait() and signal() operations for process synchronization.

Types:

- **Binary semaphore:** Values 0 or 1 (mutual exclusion)
- **Counting semaphore:** Non-negative integer for resource pools

Advantages: Prevent race conditions, ensure mutual exclusion, machine-independent, support multiple processes.

Limitations (Critical for interviews):

- Priority inversion: high-priority process waits for low-priority process
- Programming errors: incorrect wait/signal ordering causes deadlocks
- Deadlock potential: improper resource management
- Difficult to debug in large systems
- No direct support for condition synchronization

TL;DR: Semaphores = synchronization counters with wait/signal. Can cause deadlocks and priority inversion if misused.

Keyword/Key mappings: Atomic operations, mutual exclusion, counting semaphore, priority inversion, wait/signal

Q14: What is Inter-Process Communication (IPC)? What are Its

Mechanisms?

Answer: IPC is a mechanism enabling processes to communicate and synchronize.

Mechanisms include pipes, message queues, shared memory, semaphores, and sockets.

Polished Answer: IPC allows processes to exchange data and coordinate activities. Key mechanisms:

Mechanism	Communication	Speed	Use Case
Pipes	One-way, related processes	Fast	Command-line piping
Named Pipes	Unrelated processes	Fast	Cross-process streaming
Message Queues	Message-based	Medium	Asynchronous communication
Shared Memory	Direct memory access	Fastest	Large data sharing
Semaphores	Synchronization only	Fast	Resource access control
Sockets	Network communication	Slowest	Client-server, distributed systems

Choosing: Shared memory for speed (needs synchronization), message queues for structure, sockets for network.

TL;DR: IPC = processes communicating. Mechanisms: pipes, queues, shared memory, sockets, semaphores.

Keyword/Key mappings: Shared memory vs message passing, synchronization, pipe, socket, message queue

Q15: What is Demand Paging? How Does it Work?

Answer: Demand paging loads pages into memory only when they are referenced, rather than loading the entire process at startup.

Polished Answer: Demand paging is a virtual memory technique where pages are loaded on-demand, not all at once. This enables:

- Programs larger than physical memory
- Efficient memory utilization
- Reduced startup time

Working:

1. Process references a page
2. Check page table: is page in memory?
3. If valid (in memory): continue execution
4. If invalid: trigger page fault
5. OS loads page from disk into free frame
6. Update page table
7. Restart interrupted instruction

TL;DR: Demand paging = load pages only when needed. Page fault triggers loading from disk.

Keyword/Key mappings: Page fault, page table, lazy loading, virtual memory, frame allocation

Q16: What is the Banker's Algorithm?

Answer: Banker's Algorithm is a deadlock avoidance algorithm used by the OS to allocate resources safely by checking whether allocation leaves system in a safe state.

Polished Answer: Banker's Algorithm (by Dijkstra) prevents deadlock by only granting resource requests that keep the system in a safe state.

Requirements:

- Each process declares maximum resource needs
- System maintains available, allocated, and need matrices
- Before granting request, checks if safe sequence exists
- Safe state: exists sequence where all processes complete

Limitations: Requires fixed resources and known maximum demands (impractical in real systems). Used mainly for teaching and theoretical analysis.

TL;DR: Banker's Algorithm = check if resource allocation leads to safe state before granting. Prevents deadlock.

Keyword/Key mappings: Deadlock avoidance, safe state, resource allocation, maximum demands, safe sequence

Q17: What is the Difference Between Multiprogramming, Multitasking, and Multiprocessing?

Answer: Multiprogramming keeps multiple programs in memory. Multitasking shares CPU time among tasks. Multiprocessing uses multiple CPUs.

Polished Answer:

Aspect	Multiprogramming	Multitasking (Time-sharing)	Multiprocessing
CPU	Single CPU	Single CPU	Multiple CPUs
Goal	Maximize CPU utilization	Responsive interaction	Parallel execution
Switching	When process waits (I/O)	Time quantum expiry	True parallelism
User experience	Batch-oriented	Interactive	High performance
Example	Early mainframes	Windows, Linux	Multi-core servers

Modern systems combine all three: multiprogramming + time-sharing on multi-core systems.

TL;DR: Multiprogramming = CPU utilization. Multitasking = responsiveness. Multiprocessing = parallel performance.

Keyword/Key mappings: CPU utilization, time quantum, parallelism, batch processing, time-sharing

Q18: What is a Time-Sharing System?

Answer: A time-sharing system allows multiple users or processes to share the CPU by allocating a small time slice to each process, creating the illusion of simultaneous execution.

Polished Answer: Time-sharing (multitasking) systems allocate CPU in small time slices (time quantum) to multiple processes, rapidly switching between them. This provides:

- Interactive computing experience
- Multiple users on one system
- Fast response times

Key characteristics:

- CPU scheduling with time quantum
- Context switching overhead
- Process isolation and protection
- Fair resource allocation

Example: Multiple SSH sessions on a Linux server, multiple apps on Windows.

TL;DR: Time-sharing = rapid CPU switching between processes for interactive response.

Keyword/Key mappings: Time quantum, context switching, interactive computing, CPU scheduling, round robin

Q19: What is Fragmentation? Types of Fragmentation?

Answer: Fragmentation is a condition where memory becomes divided into small, scattered free blocks after repeated allocation and deallocation, resulting in inefficient memory utilization.

Polished Answer: Fragmentation wastes memory in two ways:

Internal Fragmentation:

- Occurs in fixed-size allocation (paging)
- Allocated memory exceeds needed memory
- Unused space inside allocated block
- Example: 50KB process in 64KB frame wastes 14KB

External Fragmentation:

- Occurs in variable-size allocation (segmentation)
- Free memory scattered in small holes
- Total free memory sufficient, but not contiguous
- Solutions: Compaction, paging

TL;DR: Internal = wasted space inside blocks. External = scattered free space between blocks.

Keyword/Key mappings: Memory allocation, compaction, paging, segmentation, wasted memory

Q20: What is a System Call? Why is It Needed?

Answer: A system call is a mechanism allowing user programs to request privileged services from the kernel. Needed because user programs cannot directly access hardware or protected resources.

Polished Answer: System calls provide the interface between user space and kernel space. User programs run in restricted user mode; system calls transition to kernel mode for privileged operations.

Examples:

- Process control: `fork()`, `exec()`, `exit()`
- File operations: `open()`, `read()`, `write()`, `close()`
- Device management: `ioctl()`, read/write to devices
- Communication: `socket()`, `pipe()`, `mmap()`

Process: User program → System call (software interrupt/trap) → Kernel mode → Execute service → Return to user mode

TL;DR: System calls = user programs requesting kernel services. Bridge between user and kernel mode.

Keyword/Key mappings: Kernel mode vs user mode, software interrupt, privileged operations, API, trap

Q21: What is the Producer-Consumer Problem?

Answer: A classic synchronization problem where a producer generates data into a shared buffer and a consumer removes data, with constraints on buffer full/empty conditions.

Polished Answer: The Bounded-Buffer Problem demonstrates synchronization between processes sharing a fixed-size buffer.

Constraints:

- Producer cannot add when buffer is full
- Consumer cannot remove when buffer is empty
- Only one process accesses buffer at a time

Solution using Semaphores:

- mutex: binary semaphore for buffer access (initialized to 1)
 - empty: counting semaphore for empty slots (initialized to buffer size)
 - full: counting semaphore for filled slots (initialized to 0)
-

This demonstrates mutual exclusion, synchronization, and deadlock avoidance in concurrent programming.

TL;DR: Producer-Consumer = classic sync problem with shared buffer. Solved with semaphores/mutex.

Keyword/Key mappings: Bounded buffer, race condition, semaphores, mutual exclusion, synchronization

Q22: What is Round Robin Scheduling? How Does Time Quantum Affect Performance?

Answer: Round Robin is a preemptive scheduling algorithm where each process gets a fixed time quantum in cyclic order. The time quantum size significantly impacts system performance.

Polished Answer: Round Robin ensures fairness by giving each process equal CPU time in rotation.

Time Quantum Impact:

- **Too small:** Excessive context switching overhead, CPU wasted on switching
- **Too large:** Degrades to FCFS, poor response time
- **Optimal:** Usually 10-100ms, balanced between response and overhead

Characteristics:

- Preemptive, cyclic
- No starvation (every process gets CPU)
- Good for time-sharing systems
- Average waiting time can be high for CPU-bound processes

TL;DR: RR = fixed time slice per process, cycling. Quantum size balances context switch overhead vs responsiveness.

Keyword/Key mappings: Time quantum, context switching, preemptive scheduling, fairness, starvation-free

Q23: What is the Purpose of an Operating System?

Answer: An OS is system software acting as an interface between user and hardware,

managing hardware resources and providing a convenient environment for applications.

Polished Answer: The OS serves as both resource manager and abstraction layer:

Resource Management:

- CPU scheduling among processes
- Memory allocation and protection
- File system and storage management
- I/O device coordination
- Network communication

User/Application Interface:

- Provides consistent API (system calls)
- Hides hardware complexity
- Enables concurrent execution
- Ensures security and isolation
- Provides error detection and recovery

Core Goals: Efficiency (resource utilization), Convenience (ease of use), Security (protection).

TL;DR: OS = resource manager + interface between hardware and users/applications.

Keyword/Key mappings: Resource management, hardware abstraction, system interface, process management, security

Q24: What is Spooling?

Answer: Spooling (Simultaneous Peripheral Operations On-Line) is a technique where data is temporarily stored in a buffer (usually disk) before being sent to an I/O device.

Polished Answer: Spooling decouples fast CPU operations from slow I/O devices by using disk as a buffer:

How it works:

1. Data for I/O device is written to a spool file on disk
2. Device (e.g., printer) reads from spool at its own pace
3. CPU continues processing other tasks

Benefits:

- CPU and I/O devices work concurrently
- Multiple jobs can queue for one device
- Handles speed differences between CPU and peripherals
- Example: Print spooler in Windows/Linux

TL;DR: Spooling = buffering data on disk for slow devices. CPU and I/O work in parallel.

Keyword/Key mappings: Print spooler, I/O buffering, concurrent operation, device management, disk buffer

Q25: What is the Difference Between a System Call and a Library Call?

Answer: A system call is a request to the kernel for privileged operations. A library call is a function provided by a programming library that may or may not invoke system calls.

Polished Answer:

Aspect	System Call	Library Call
Execution	Kernel mode	User mode
Overhead	High (mode switch)	Low (direct function call)
Provider	OS kernel	Language/runtime library
Examples	open(), read(), write()	printf(), malloc(), strcpy()
Privilege	Requires kernel	No kernel involvement

Relationship: Library calls often wrap system calls. Example: printf() (library) internally calls write() (system call). Some library calls like strcpy() don't use system calls at all.

TL;DR: System call = kernel request (slow). Library call = user-space function (fast, may call system calls).

Keyword/Key mappings: User mode vs kernel mode, libc, function call overhead, API layers, wrapper functions

TOP 50 QUESTIONS (26-50)

Q26: What are Orphan and Zombie Processes?

Answer: An orphan process is a child whose parent terminated before it. A zombie process has completed execution but still has an entry in the process table because its parent hasn't collected its exit status.

Polished Answer:

Orphan Process:

- Parent terminates before child
- Adopted by init/systemd (PID 1)
- Continues execution normally
- init reaps it when it finishes

Zombie Process:

- Process has terminated but not reaped
- Parent hasn't called wait()
- Occupies only PCB entry, no CPU/memory
- Cannot be killed (already dead)
- Too many zombies exhaust process table

Prevention: Parent must call wait()/waitpid() to collect child exit status. Or use SIGCHLD handler or double-fork technique.

TL;DR: Orphan = parent died first, adopted by init. Zombie = process finished but parent hasn't collected exit status.

Keyword/Key mappings: Process termination, exit status, wait() system call, init process, process table

Q27: What is Starvation and Aging?

Answer: Starvation occurs when a process waits indefinitely for resources because higher-priority processes keep getting served. Aging prevents starvation by gradually increasing priority of waiting processes.

Polished Answer:

Starvation:

- Low-priority processes never get CPU/resources
- Common in priority scheduling and SJF
- Can also occur with unfair synchronization

Aging:

- Gradually increases priority of waiting processes
- Each waiting period adds to priority
- Eventually, even lowest priority reaches top
- Ensures bounded waiting and fairness

Example: In priority scheduling, a process with priority 5 (low) might gain +1 priority every minute, eventually reaching priority 10 and getting scheduled.

TL;DR: Starvation = indefinite waiting. Aging = gradually boosting priority to prevent starvation.

Keyword/Key mappings: Priority scheduling, fairness, bounded waiting, resource allocation, process scheduling

Q28: What is Cache Memory and Its Working?

Answer: Caching stores frequently accessed data in small, high-speed memory to reduce average memory access time.

Polished Answer: Cache memory is a small, fast memory between CPU and main memory (RAM). It exploits locality of reference:

Types of Locality:

- Temporal: Recently used data likely used again
- Spatial: Nearby memory locations likely accessed

Cache Hierarchy:

- L1 Cache: Smallest, fastest (inside CPU core)
- L2 Cache: Larger, slightly slower
- L3 Cache: Shared among cores, larger still
- Main Memory (RAM): Much larger, much slower

Performance: Cache hit (data in cache) is fast. Cache miss requires fetching from slower memory. Hit rate determines effectiveness.

TL;DR: Cache = small fast memory storing frequently used data. Works via temporal and

spatial locality.

Keyword/Key mappings: Cache hit/miss, L1/L2/L3 cache, locality of reference, memory hierarchy, access time

Q29: What is RAID? Different RAID Levels?

Answer: RAID (Redundant Array of Independent Disks) combines multiple disks for improved performance, reliability, and capacity.

Polished Answer: RAID provides data redundancy and performance through disk arrays:

RAID Level	Technique	Min Disks	Fault Tolerance	Use Case
RAID 0	Striping	2	None	Performance only
RAID 1	Mirroring	2	1 disk failure	Reliability
RAID 5	Striping + Distributed Parity	3	1 disk failure	Balanced
RAID 6	Striping + Double Parity	4	2 disk failures	High reliability
RAID 10	Mirror + Stripe	4	1 per mirror pair	Performance + Reliability

Trade-offs: RAID 0 (speed, no safety), RAID 1 (safety, half capacity), RAID 5 (balance), RAID 6 (maximum safety).

TL;DR: RAID = multiple disks for performance/reliability. RAID 0 (speed), RAID 1 (mirror), RAID 5 (parity), RAID 6 (double parity).

Keyword/Key mappings: Data redundancy, striping, mirroring, parity, fault tolerance

Q30: What is the Dispatcher and Dispatch Latency?

Answer: A dispatcher is the OS module that transfers CPU control to the process selected by the scheduler. Dispatch latency is the time to stop one process and start another.

Polished Answer:

Dispatcher Functions:

- Context switching (save/restore state)

- Switching from kernel mode to user mode
- Jumping to the correct instruction in the new process

Dispatch Latency Components:

- Time to save current process state
- Time to select next process (scheduler decision)
- Time to load new process state
- Total time system is in "overhead" mode

Importance: Critical for real-time systems where response time matters. Lower dispatch latency = faster system response.

TL;DR: Dispatcher = hands CPU to scheduled process. Dispatch latency = time between processes executing.

Keyword/Key mappings: Context switch, scheduler, kernel mode, real-time systems, process state

Q31: What is the Critical Section Problem?

Answer: A critical section is a code segment where shared resources are accessed. Only one process/thread should enter at a time to prevent race conditions.

Polished Answer: The Critical Section Problem addresses safe access to shared data in concurrent programs.

Requirements for Solution:

1. **Mutual Exclusion:** Only one process in critical section at a time
2. **Progress:** If no one in CS and processes want entry, decision must be made promptly
3. **Bounded Waiting:** No process waits indefinitely (fairness)

Solutions:

- Software: Peterson's Algorithm (2 processes), Dekker's Algorithm
- Hardware: Test-and-Set, Compare-and-Swap instructions
- OS: Semaphores, Mutexes, Monitors

TL;DR: Critical section = code accessing shared data. Needs mutual exclusion, progress, bounded waiting.

Keyword/Key mappings: Race condition, mutual exclusion, synchronization, test-and-set, Peterson's algorithm

Q32: What is Peterson's Algorithm?

Answer: Peterson's Algorithm is a software-based synchronization algorithm providing mutual exclusion for two processes using flag array and turn variable.

Polished Answer: Peterson's solution uses two shared variables:

- flag[i]: indicates process i wants to enter critical section
- turn: determines which process gets priority

Algorithm:

text

```
javascript.snippet                                JAVASCRIPT

flag[i] = true; turn = j; while (flag[j] && turn == j); // wait // Critical Section
flag[i] = false; // exit
```

Properties: Ensures mutual exclusion, progress, and bounded waiting for exactly two processes. Fails in modern multi-core systems due to memory ordering issues (needs memory barriers).

TL;DR: Peterson's = 2-process mutual exclusion using flags and turn variable. Software-only solution.

Keyword/Key mappings: Mutual exclusion, flag array, turn variable, busy waiting, synchronization

Q33: What is the Difference Between Logical and Physical Address?

Answer: Logical address is generated by CPU (virtual address). Physical address is the actual address in main memory (RAM).

Polished Answer:

Aspect	Logical Address	Physical Address
Generation	CPU generates	MMU computes from logical
Visibility	Visible to programmer	Hidden from programmer
Space	Virtual address space per process	Physical memory (RAM)
Size	Can exceed physical memory	Limited by RAM size
Translation	Requires MMU + page table	Direct hardware access

Process: CPU generates logical address MMU translates Physical address in RAM. This enables virtual memory and process isolation.

TL;DR: Logical = CPU-generated virtual address. Physical = actual RAM address after MMU translation.

Keyword/Key mappings: MMU, address translation, page table, virtual memory, RAM

Q34: What is Memory Protection and How is It Achieved?

Answer: Memory protection prevents one process from accessing or modifying memory allocated to another process, ensuring system stability and security.

Polished Answer: Memory protection is enforced through:

- **Virtual Memory:** Each process has own address space
- **Page Tables:** Map virtual to physical with permission bits (read/write/execute)
- **MMU:** Hardware-enforced access control
- **Kernel Mode vs User Mode:** Only kernel can access protected regions
- **Segmentation:** Segment limits prevent out-of-bounds access

This ensures program crashes don't affect other processes and security boundaries are maintained.

TL;DR: Memory protection = process isolation via virtual memory, page table permissions, and hardware enforcement.

Keyword/Key mappings: Page table permissions, MMU, address space isolation, segmentation fault, kernel mode

Q35: What is a Page Fault and How is It Handled?

Answer: A page fault occurs when a process references a page not currently in physical memory. The OS must load the page from disk.

Polished Answer: Page fault handling sequence:

1. CPU references a virtual address
2. Page table lookup: page not in memory (invalid bit set)
3. Trap to OS (page fault handler)
4. Validate reference (check if legal address)
5. Find free frame in physical memory
6. If no free frame, evict a page (page replacement)
7. Load required page from disk into frame
8. Update page table
9. Restart the faulting instruction

Page faults are essential for demand paging but cause significant delay (disk I/O).

TL;DR: Page fault = referenced page not in RAM. OS loads it from disk, updates page table, restarts instruction.

Keyword/Key mappings: Demand paging, page table, disk I/O, page replacement, virtual memory

Q36: What is the Working Set Model?

Answer: The working set is the set of pages a process is actively using in a recent time window. It helps determine how many frames a process needs to avoid thrashing.

Polished Answer: Working Set Model:

- **Working Set Window ():** Recent time interval (e.g., last 10,000 references)
- **Working Set Size:** Number of unique pages referenced in window
- **Principle:** Process needs its working set in memory to run efficiently

Application:

- If total working sets > available memory thrashing imminent
- OS suspends processes to reduce memory pressure
- Dynamic frame allocation based on working set size

TL;DR: Working set = pages actively used recently. If total exceeds RAM, thrashing occurs.

Keyword/Key mappings: Thrashing prevention, page references, frame allocation, locality of

Q37: What is a Monitor?

Answer: A monitor is a high-level synchronization construct combining mutual exclusion with condition variables for safe access to shared resources.

Polished Answer: Monitors encapsulate shared data with operations, providing:

- **Automatic mutual exclusion:** Only one thread executes monitor code at a time
- **Condition variables:** Allow threads to wait and be notified
- **Simpler usage:** No explicit locking required

Operations:

- `wait(condition)`: Release monitor, block until signaled
- `signal(condition)`: Wake up one waiting thread
- `broadcast(condition)`: Wake up all waiting threads

Languages: Java (synchronized methods), C# (lock), Python (with `threading.Condition`).

TL;DR: Monitor = high-level synchronization with automatic locking and condition variables.

Keyword/Key mappings: Condition variables, mutual exclusion, wait/signal, synchronized methods, thread safety

Q38: What is Priority Inversion?

Answer: Priority inversion occurs when a high-priority process is blocked by a low-priority process holding a required resource, while a medium-priority process preempts the low-priority one.

Polished Answer: Priority inversion can cause critical system failures (famously on Mars Pathfinder).

Scenario:

1. Low-priority process L holds resource R
2. High-priority process H requests R, blocks
3. Medium-priority process M preempts L
4. H waits for M to finish, then L to finish indefinite delay

Solutions:

- **Priority Inheritance:** L temporarily inherits H's priority while holding R
- **Priority Ceiling:** Each resource has max priority; holder runs at that priority

TL;DR: Priority inversion = high-priority blocked by low-priority. Fix: priority inheritance or ceiling.

Keyword/Key mappings: Priority inheritance, priority ceiling, real-time systems, resource lock, Mars Pathfinder

Q39: What is the Readers-Writers Problem?

Answer: A synchronization problem managing access to shared data where multiple readers can read simultaneously, but writers need exclusive access.

Polished Answer: The Readers-Writers Problem allows concurrent reads but exclusive writes.

Constraints:

- Multiple readers can access simultaneously
- Writers must have exclusive access
- No reader while writer is active

Solutions:

- **Readers-preference:** New readers proceed even if writer waiting (writer starvation possible)
- **Writers-preference:** Writers get priority (reader starvation possible)
- **Fair solution:** FIFO ordering prevents starvation

Implementation: Usually uses mutex for read count, semaphore for write access.

TL;DR: Readers-Writers = concurrent reads allowed, exclusive writes. Balance between reader and writer priority.

Keyword/Key mappings: Shared data access, reader-writer lock, mutual exclusion, starvation, database access

Q40: What is the Dining Philosophers Problem?

Answer: A classic synchronization problem demonstrating deadlock when multiple processes compete for shared resources.

Polished Answer: Five philosophers sit around a table with five forks (one between each pair). To eat, a philosopher needs both adjacent forks. Challenges:

- If all pick up left fork simultaneously deadlock
- Resource contention and synchronization

Solutions:

- Limit philosophers at table (allow 4 at most)
- Pick forks in order (prevent circular wait)
- Pick both forks simultaneously (atomic operation)
- Asymmetric picking (odd pick left first, even pick right first)

TL;DR: Dining Philosophers = resource allocation problem demonstrating deadlock avoidance.

Keyword/Key mappings: Deadlock prevention, circular wait, mutual exclusion, resource contention

Q41: What is the Difference Between Mutex and Semaphore?

Answer: A mutex is a locking mechanism for mutual exclusion (one thread at a time). A semaphore is a signaling mechanism controlling access to a pool of resources.

Polished Answer:

Aspect	Mutex	Semaphore
Purpose	Mutual exclusion	Signaling/controlling access
Value	Binary (locked/unlocked)	Integer (count resources)
Ownership	Owner must release	Any thread can signal
Recursion	Can be recursive (some impl)	Not recursive
Resource count	Protects single resource	Protects N resources
Analogy	Key to one room	Traffic signal

Use: Mutex for critical sections. Semaphore for resource pools (database connections) or signaling.

TL;DR: Mutex = lock for one resource. Semaphore = counter for N resources. Mutex has ownership, semaphore doesn't.

Keyword/Key mappings: Binary semaphore vs mutex, ownership, counting semaphore, critical section, signaling

Q42: What are Page Replacement Algorithms?

Answer: Algorithms deciding which page to evict from memory when a new page needs to be loaded and no free frames exist.

Polished Answer:

Algorithm	Description	Pros	Cons
FIFO	Evict oldest page	Simple	Belady's anomaly, poor performance
LRU	Evict least recently used	Good performance	Hard to implement perfectly
Optimal	Evict page not needed longest	Best possible	Not implementable (future unknown)
Clock/Second-Chance	Approximate LRU	Practical, efficient	Not as good as true LRU

Implementation: LRU approximated using reference bits (Clock algorithm). Optimal used as benchmark.

TL;DR: Page replacement = choosing victim page. FIFO (oldest), LRU (least used), Optimal (ideal, theoretical), Clock (practical).

Keyword/Key mappings: Belady's anomaly, reference bits, victim page, frame allocation, page eviction

Q43: What is the Difference Between Buffering and Caching?

Answer: Buffering temporarily stores data during transfer between devices. Caching stores frequently accessed data for faster subsequent access.

Polished Answer:

Aspect	Buffering	Caching
Purpose	Handle speed differences	Speed up access
Data	Data in transit	Frequently used data
Duration	Temporary, cleared	Persistent until evicted
Location	Between producer/consumer	Close to CPU or device
Example	Print buffer, I/O buffer	CPU cache, web cache

Both use fast storage to bridge speed gaps, but buffering is for transfer, caching is for reuse.

TL;DR: Buffer = temporary storage during transfer. Cache = store frequently used data for faster access.

Keyword/Key mappings: I/O performance, speed mismatch, cache hit rate, data transfer, memory hierarchy

Q44: What is the Difference Between Hard Real-Time and Soft Real-Time Systems?

Answer: Hard real-time systems have strict deadlines that must be met. Soft real-time systems have deadlines that should be met but occasional misses are acceptable.

Polished Answer:

Aspect	Hard Real-Time	Soft Real-Time
Deadline	Miss = system failure	Miss = degraded quality
Consequences	Catastrophic	Acceptable
Examples	Airbag control, pacemaker	Video streaming, games
Scheduling	Deterministic	Best effort
Latency	Guaranteed	Aimed for, not guaranteed

Hard real-time requires careful analysis and guaranteed resource allocation.

TL;DR: Hard RT = deadlines mandatory (safety systems). Soft RT = deadlines preferred (media applications).

Keyword/Key mappings: Deadline miss, deterministic scheduling, embedded systems,

Q45: What is the Difference Between Monolithic and Microkernel?

Answer: Monolithic kernels run all services in kernel space. Microkernels run minimal services in kernel space, most services in user space.

Polished Answer:

Aspect	Monolithic Kernel	Microkernel
Services	All in kernel space	Minimal in kernel, rest in user space
Performance	Faster (direct calls)	Slower (IPC between services)
Size	Large	Small
Reliability	Service crash = system crash	Service crash isolated
Extensibility	Harder	Easier (modular)
Examples	Linux, Unix	QNX, Minix

Modern kernels are hybrid: monolithic structure with modular design (loadable kernel modules).

TL;DR: Monolithic = all services in kernel (fast, bigger). Microkernel = minimal kernel (safer, slower).

Keyword/Key mappings: Kernel space vs user space, IPC overhead, modular design, system reliability, performance

Q46: What is a Thread Pool and Why Use It?

Answer: A thread pool is a collection of pre-created threads ready to execute tasks, avoiding the overhead of creating threads dynamically.

Polished Answer: Thread pools manage a fixed set of worker threads that process queued tasks:

Benefits:

- Avoid thread creation/destruction overhead
- Control resource usage (limit concurrent threads)
- Better load balancing
- Improved response time (threads ready immediately)

Implementation: Queue of tasks, N worker threads consume from queue. When task done, thread returns to pool.

Configuration: Pool size depends on CPU-bound (n = cores) vs I/O-bound (n = more threads) tasks.

TL;DR: Thread pool = pre-created threads processing task queue. Reduces overhead, controls concurrency.

Keyword/Key mappings: Task queue, worker threads, thread lifecycle, concurrency management, resource control

Q47: What is Copy-on-Write?

Answer: Copy-on-Write (COW) is a memory optimization where multiple processes share the same memory pages until one modifies them, triggering a copy.

Polished Answer: COW defers memory copying until actually needed:

How it works:

1. Parent and child share same physical pages
2. Pages marked read-only
3. If any process writes page fault
4. OS copies page, makes writable copy for writer
5. Original shared page remains for others

Applications:

- fork() system call (process creation)
- Virtual machine snapshotting
- File system snapshots

Benefit: Avoids unnecessary copying, saves memory and time.

TL;DR: COW = share memory until write occurs, then copy. Optimizes fork() and snapshots.

Keyword/Key mappings: Fork system call, page fault, memory optimization, process

Q48: What is the Difference Between Compile-Time, Load-Time, and Execution-Time Address Binding?

Answer: Address binding determines when logical addresses are mapped to physical addresses: at compile, load, or execution time.

Polished Answer:

Binding Time	Description	Flexibility	Example
Compile-time	Physical addresses known at compile	None (must reload at same location)	Embedded systems
Load-time	Addresses assigned when program loaded	Can load at different locations	Relocatable code
Execution-time	Addresses assigned during execution	Maximum (can move process)	Virtual memory systems

Execution-time binding enables virtual memory and process migration.

TL;DR: Address binding = when virtual maps to physical. Compile (fixed), Load (relocatable), Execution (dynamic, most flexible).

Keyword/Key mappings: Relocation, virtual memory, MMU, process loading, address translation

Q49: What is File Allocation Table (FAT)?

Answer: FAT is a disk data structure tracking file locations and cluster allocation on storage devices.

Polished Answer: FAT maintains a table of entries, one per cluster, indicating:

- Next cluster in file chain
- End of file (EOF) marker
- Bad cluster marker
- Free cluster marker

Characteristics:

- Simple and widely supported
- File traversal follows cluster chain
- Fragmentation possible
- Used in FAT12, FAT16, FAT32 file systems

Limitations: No journaling, limited file sizes (FAT32: 4GB max file), slower for random access.

TL;DR: FAT = table mapping clusters to files. Simple but lacks journaling and large file support.

Keyword/Key mappings: Cluster allocation, file system, disk organization, directory entry, file chain

Q50: What is an Inode?

Answer: An inode is a data structure storing metadata about a file (not the file content or name).

Polished Answer: Each file in Unix-like systems has an inode containing:

- File size
- Ownership (user, group)
- Permissions (read/write/execute)
- Timestamps (create, modify, access, change)
- Link count (hard links)
- Pointers to data blocks

Key Point: File names are separate from inodes. Directories map names to inode numbers. Multiple names (hard links) can point to same inode.

TL;DR: Inode = file metadata structure (size, permissions, timestamps, data block pointers). Name stored separately.

Keyword/Key mappings: Hard link vs soft link, metadata, data blocks, directory entry, Unix file system

TOP 100 QUESTIONS (51-100)

Q51: What is the Difference Between Pipes and Sockets?

Answer: Pipes enable communication between related processes on the same machine. Sockets enable communication between processes on the same or different machines.

Polished Answer:

Aspect	Pipes	Sockets
Scope	Same machine, related processes	Any machines (network)
Communication	Unidirectional (named pipes bidirectional)	Bidirectional
Protocol	No network protocol	TCP/IP, UDP, Unix domain
Setup	Simple	More complex
Use case	Command-line piping	Client-server, distributed systems

Pipes are for local IPC; sockets for network communication (though Unix domain sockets work locally too).

TL;DR: Pipes = local, related processes. Sockets = network, any processes. Sockets are more flexible.

Keyword/Key mappings: IPC, TCP/IP, named pipe, Unix domain socket, network communication

Q52: What is a Named Pipe (FIFO)?

Answer: A named pipe is a pipe with a filesystem name, allowing communication between unrelated processes.

Polished Answer: Named pipes (FIFOs) extend pipe functionality:

Differences from Anonymous Pipes:

- Have a pathname in filesystem
- Can be used by unrelated processes
- Persist until explicitly deleted
- Support bidirectional communication (implementation-dependent)

Usage:

bash

```
javascript.snippet JAVASCRIPT

mkfifo mypipe# Process 1: echo "data" > mypipe# Process 2: cat < mypipe
```

TL;DR: Named pipe = pipe with filesystem name, works between unrelated processes.

Keyword/Key mappings: FIFO, filesystem, IPC, unrelated processes, mkfifo

Q53: What is Memory-Mapped I/O?

Answer: Memory-mapped I/O maps device registers or file content into the process address space, allowing direct access as if it were memory.

Polished Answer: Memory-mapped I/O enables:

- **File access:** `mmap()` maps file content to memory for direct read/write
- **Device access:** Device registers mapped to memory addresses
- **Shared memory:** Multiple processes map same file for communication

Benefits: Fast access, automatic kernel handling, no explicit read/write calls.

Trade-offs: Complex error handling, page fault overhead.

TL;DR: Memory-mapped I/O = access files/devices via memory addresses. Faster for large data.

Keyword/Key mappings: `mmap`, virtual memory, page cache, shared memory, device registers

Q54: What is a Device Driver?

Answer: A device driver is software that enables the OS to communicate with hardware devices by translating generic OS requests into device-specific commands.

Polished Answer: Device drivers serve as intermediaries:

Functions:

- Initialize and configure hardware
- Translate OS I/O requests to device operations
- Handle device interrupts

- Manage data transfer (DMA, polling)
- Report errors and status

Types:

- Character device drivers (keyboard, serial)
- Block device drivers (disk, SSD)
- Network device drivers (Ethernet, WiFi)

TL;DR: Device driver = software translating OS commands to hardware operations.

Keyword/Key mappings: Hardware abstraction, interrupt handler, DMA, character vs block device, kernel module

Q55: What is DMA (Direct Memory Access)?

Answer: DMA allows devices to transfer data directly to/from memory without CPU involvement, improving I/O performance.

Polished Answer: DMA controller performs data transfers:

How it works:

1. CPU programs DMA controller (source, destination, size)
2. DMA controller transfers data directly between device and memory
3. DMA controller interrupts CPU when transfer complete
4. CPU continues other work during transfer

Benefits: Reduces CPU overhead, faster I/O, enables concurrent processing.

Cycle stealing: DMA briefly takes bus control from CPU for each transfer.

TL;DR: DMA = direct device-to-memory transfer without CPU. Reduces CPU I/O overhead.

Keyword/Key mappings: Bus control, cycle stealing, interrupt, I/O performance, device controller

Q56: What is a File System? Basic Operations?

Answer: A file system organizes, stores, retrieves, and manages files on storage devices.

Polished Answer: File system provides logical structure for data:

Basic Operations:

- Create: Allocate space and create directory entry
- Open: Load file metadata, prepare for access
- Read: Transfer data from disk to memory
- Write: Transfer data from memory to disk
- Append: Add data at end of file
- Delete: Free space, remove directory entry
- Truncate: Reduce file size
- Close: Finish access, release resources

Components: Directory structure, allocation methods, free space management.

TL;DR: File system = organizing and managing files. Operations: create, open, read, write, delete, close.

Keyword/Key mappings: Directory structure, file allocation, metadata, storage management, file operations

Q57: What is the Difference Between Sequential and Direct Access?

Answer: Sequential access reads/writes data in order (one after another). Direct access can read/write any location immediately.

Polished Answer:

Aspect	Sequential Access	Direct Access
Order	Must access in sequence	Any order
Speed	Slower for random access	Fast for any access
Storage	Tapes	Disks, SSD
Use case	Backups, streaming	Databases, random read/write

Modern file systems support both: sequential file reading and direct (random) access via seek operations.

TL;DR: Sequential = in order. Direct = any location immediately. Disks support both.

Keyword/Key mappings: Random access, seek operation, file access methods, storage media

Q58: What is Seek Time and Rotational Latency?

Answer: Seek time is the time to move the disk arm to the correct track. Rotational latency is the time for the desired sector to rotate under the read/write head.

Polished Answer: Both are components of disk access time:

Seek Time:

- Time to move read/write head to correct track
- Depends on distance arm must travel
- Usually the largest component of disk access time

Rotational Latency:

- Time for disk to rotate desired sector under head
- Average = half rotation time
- Depends on disk RPM (faster = lower latency)

Total access time = Seek time + Rotational latency + Transfer time

TL;DR: Seek time = move head to track. Rotational latency = wait for sector to rotate under head.

Keyword/Key mappings: Disk access time, read/write head, tracks, sectors, disk RPM

Q59: What are Disk Scheduling Algorithms?

Answer: Algorithms determining the order in which disk I/O requests are serviced to minimize seek time and improve performance.

Polished Answer:

Algorithm	Description	Pros	Cons
FCFS	Service in arrival order	Fair, simple	Poor performance
SSTF	Service closest request first	Better seek time	Starvation possible
SCAN (Elevator)	Sweep back and forth	No starvation	Not uniform

C-SCAN	Circular sweep, one direction	More uniform	Wastes time
LOOK/C-LOOK	Like SCAN but stops at last request	More efficient	Still directional

Modern systems use variants of SCAN/LOOK for balanced performance.

TL;DR: Disk scheduling = optimizing seek order. SSTF (shortest), SCAN (sweep), LOOK (efficient sweep).

Keyword/Key mappings: Seek time optimization, starvation, elevator algorithm, disk I/O, request queue

Q60: What is the Difference Between SSD and HDD?

Answer: SSD (Solid State Drive) uses flash memory with no moving parts. HDD (Hard Disk Drive) uses magnetic spinning disks with read/write heads.

Polished Answer:

Aspect	SSD	HDD
Technology	Flash memory (NAND)	Magnetic spinning disks
Speed	Very fast (no seek time)	Slower (seek + rotational latency)
Durability	Better (no moving parts)	Susceptible to shock
Price per GB	Higher	Lower
Lifespan	Limited write cycles	Mechanical wear
Fragmentation	Doesn't matter	Affects performance

OS treats both as block devices, but scheduling differs (no seek optimization needed for SSD).

TL;DR: SSD = flash, faster, no moving parts. HDD = magnetic, cheaper per GB, has seek time.

Keyword/Key mappings: Flash memory, block device, wear leveling, seek time, storage performance

Q61: What is an Interrupt?

Answer: An interrupt is a signal generated by hardware or software that temporarily pauses the current task to execute an Interrupt Service Routine (ISR).

Polished Answer: Interrupts enable responsive handling of events:

Types:

- **Hardware interrupts:** From devices (keyboard, timer, disk)
- **Software interrupts (traps):** From programs (system calls, exceptions)
- **Maskable vs Non-maskable:** Can be disabled vs critical

Process:

1. Interrupt signal received
2. Current state saved
3. ISR executes
4. State restored, execution resumes

TL;DR: Interrupt = signal for immediate CPU attention. Executes ISR, then resumes.

Keyword/Key mappings: ISR, interrupt vector, hardware vs software interrupt, interrupt priority, NMI

Q62: What is a Trap?

Answer: A trap is a software-generated interrupt occurring due to an exception, error, or system call, transferring control to the OS.

Polished Answer: Traps are synchronous events:

Causes:

- System calls (intentional)
- Division by zero
- Invalid memory access
- Illegal instruction

Difference from interrupt: Interrupts are asynchronous (external), traps are synchronous (instruction-caused).

TL;DR: Trap = software interrupt from program (system call or error). Synchronous,

intentional or error.

Keyword/Key mappings: Software interrupt, system call, exception handling, synchronous event, kernel mode

Q63: What is the Bootstrapping Process?

Answer: Bootstrapping (booting) is the process of loading and initializing the OS when a computer starts.

Polished Answer: Boot sequence:

1. **Power-on:** CPU starts executing BIOS/UEFI firmware
2. **POST:** Power-On Self-Test checks hardware
3. **Bootloader:** BIOS loads bootloader from boot sector
4. **Kernel loading:** Bootloader loads kernel into memory
5. **Kernel initialization:** Sets up drivers, memory, processes
6. **Init process:** Starts system services (init/systemd)
7. **User login:** System ready for use

TL;DR: Booting = firmware bootloader kernel init system ready.

Keyword/Key mappings: BIOS, UEFI, bootloader, kernel loading, POST, init process

Q64: What is a Deadlock Prevention vs Avoidance vs Detection?

Answer: Prevention breaks deadlock conditions. Avoidance uses safe state checks.

Detection identifies and recovers from deadlocks.

Polished Answer:

Strategy	Approach	Method	Cost
Prevention	Break one Coffman condition	Restrict resource requests	Reduced concurrency
Avoidance	Check safety before allocation	Banker's Algorithm	Runtime overhead
Detection	Detect and recover	Resource graph cycle detection	Recovery cost

Recovery methods: Process termination, resource preemption.

TL;DR: Prevention = break conditions. Avoidance = check safety. Detection = find and recover.

Keyword/Key mappings: Coffman conditions, Banker's Algorithm, resource allocation graph, rollback, safe state

Q65: What is a Resource Allocation Graph (RAG)?

Answer: A Resource Allocation Graph represents resource allocation and requests, helping detect deadlocks.

Polished Answer: RAG components:

- **Process nodes:** Circles
- **Resource nodes:** Rectangles
- **Assignment edges:** Resource → Process (resource held)
- **Request edges:** Process → Resource (process waiting)

Deadlock detection: If graph has a cycle → possible deadlock (with single-instance resources, cycle guarantees deadlock).

TL;DR: RAG = graph showing which processes hold/wait for resources. Cycles indicate potential deadlock.

Keyword/Key mappings: Deadlock detection, cycle detection, resource allocation, process-resource graph

Q66: What is a Dispatcher and Scheduler?

Answer: Scheduler selects which process runs next. Dispatcher transfers CPU control to the selected process.

Polished Answer:

Component	Function
Long-term scheduler	Controls admission to ready queue
Short-term scheduler (CPU scheduler)	Selects next process from ready queue
Medium-term scheduler	Swaps processes in/out of memory
Dispatcher	Performs context switch to selected process

Dispatcher functions: switch context, switch to user mode, jump to correct location.

TL;DR: Scheduler = picks process. Dispatcher = actually switches CPU to that process.

Keyword/Key mappings: Long/medium/short-term scheduler, context switch, ready queue, process selection

Q67: What is a Thread Library?

Answer: A thread library provides APIs for creating, managing, and synchronizing threads in user-level programs.

Polished Answer: Common thread libraries:

- **Pthreads:** POSIX standard (Unix/Linux)
- **Windows Threads:** Win32 API
- **Java Threads:** JVM-managed
- **OpenMP:** Parallel programming

Functions:

- Thread creation/destruction
- Thread synchronization (mutex, condition variables)
- Thread scheduling hints

TL;DR: Thread library = API for thread management. Pthreads, Windows, Java threads.

Keyword/Key mappings: POSIX threads, thread API, concurrency, synchronization primitives

Q68: What is a Spinlock?

Answer: A spinlock is a synchronization mechanism where a thread repeatedly checks a

lock condition in a loop ("spins") until it becomes available.

Polished Answer: Spinlocks are busy-waiting locks:

Characteristics:

- CPU cycles wasted while waiting
- No context switch overhead
- Good for short critical sections
- Used in kernel synchronization
- Bad for long waits (CPU waste)

vs Mutex: Mutex blocks (context switch), spinlock burns CPU.

TL;DR: Spinlock = busy-waiting lock. Fast for short waits, wastes CPU for long waits.

Keyword/Key mappings: Busy waiting, kernel synchronization, context switch avoidance, lock contention

Q69: What is a Lock-Free Data Structure?

Answer: Lock-free data structures allow concurrent access without traditional locks, using atomic operations (CAS).

Polished Answer: Based on Compare-And-Swap (CAS) atomic operations:

Benefits:

- No deadlock possible
- No priority inversion
- Scalable under contention
- Progress guarantee (at least one thread progresses)

Examples: Lock-free queue, stack, linked list. Used in high-performance systems.

Challenges: Complex to implement correctly, ABA problem.

TL;DR: Lock-free = concurrency without locks, using atomic CAS. Complex but deadlock-free.

Keyword/Key mappings: CAS, atomic operations, ABA problem, concurrent data structures, scalability

Q70: What is the Difference Between Strong and Weak Consistency?

Answer: Strong consistency guarantees all processes see the same data at the same time. Weak consistency allows temporary divergence.

Polished Answer:

Aspect	Strong Consistency	Weak Consistency
Guarantee	All reads see latest write	Reads may see stale data
Performance	Slower (synchronization)	Faster
Implementation	Locks, transactions	Eventual consistency
Example	Relational databases	DNS, cache systems

Distributed systems often choose weak consistency for performance.

TL;DR: Strong = all see same data immediately. Weak = eventual consistency, faster.

Keyword/Key mappings: Distributed systems, eventual consistency, synchronization, cache coherence

Q71: What is a Race Condition?

Answer: A race condition occurs when the outcome depends on the timing or order of execution of concurrent processes/threads accessing shared data.

Polished Answer: Race conditions are classic concurrency bugs:

Example:

text

```
javascript.snippet                                JAVASCRIPT

Thread 1: x = x + 1  (reads x=5, adds 1)Thread 2: x = x + 1  (reads x=5, adds 1)Both
write
x=6 (should be 7!)
```

Prevention:

- Synchronization (mutex, semaphore)
- Atomic operations
- Lock-free data structures
- Avoiding shared mutable state

TL;DR: Race condition = outcome depends on execution timing. Fix with synchronization.

Keyword/Key mappings: Concurrent access, shared data, atomic operations, synchronization, data corruption

Q72: What is a Barrier in Concurrency?

Answer: A barrier is a synchronization point where all threads must arrive before any can continue.

Polished Answer: Barriers coordinate phase transitions in parallel programs:

How it works:

1. Threads arrive at barrier and wait
2. When all threads arrive, all are released
3. Each thread continues to next phase

Deadlock scenario: If one thread never reaches barrier (crash, infinite loop), all others wait forever.

Use cases: Parallel algorithms with distinct phases (matrix multiplication, MapReduce).

TL;DR: Barrier = all threads wait until everyone arrives. Deadlock if one thread never arrives.

Keyword/Key mappings: Phase synchronization, parallel programming, thread coordination, deadlock scenario

Q73: What is the Difference Between Concurrency and Parallelism?

Answer: Concurrency is overlapping execution (interleaving tasks). Parallelism is simultaneous execution (multiple CPUs).

Polished Answer:

Aspect	Concurrency	Parallelism
Definition	Managing multiple tasks, interleaved	Executing multiple tasks simultaneously
Hardware	Single CPU possible	Requires multiple cores
Goal	Responsiveness, resource utilization	Speedup
Example	Node.js event loop	Multi-threaded program on multi-core

Concurrency is about structure; parallelism is about hardware execution.

TL;DR: Concurrency = interleaving. Parallelism = simultaneous. Concurrency on single CPU, parallelism on multi-core.

Keyword/Key mappings: Multi-threading, multi-core, interleaving, simultaneous execution, task scheduling

Q74: What is the ABA Problem?

Answer: The ABA problem occurs in lock-free programming when a value changes from A to B and back to A, fooling CAS operations into thinking nothing changed.

Polished Answer: ABA problem scenario:

1. Thread reads value A
2. Another thread changes A B A
3. First thread's CAS succeeds (sees A, thinks unchanged)
4. But intermediate state changed!

Solutions:

- Version/tag bits (increment counter on each modification)
- Double-word CAS
- Hazard pointers

TL;DR: ABA = value changes and returns, CAS falsely thinks no change. Fix with version numbers.

Keyword/Key mappings: CAS, lock-free programming, version tags, atomic operations, memory management

Q75: What is Memory Management Unit (MMU)?

Answer: MMU is hardware that translates virtual (logical) addresses to physical addresses and enforces memory protection.

Polished Answer: MMU functions:

- **Address translation:** Virtual Physical using page table
- **Memory protection:** Enforce read/write/execute permissions
- **Cache control:** Manage TLB for fast translation

Process: CPU sends virtual address MMU checks TLB if miss, walk page table return physical address.

TL;DR: MMU = hardware translating virtual to physical addresses. Enforces memory protection.

Keyword/Key mappings: Page table, TLB, address translation, memory protection, virtual memory

Q76: What is a System Process vs User Process?

Answer: System processes execute in kernel mode with full privileges. User processes execute in user mode with restricted access.

Polished Answer:

Aspect	System Process	User Process
Mode	Kernel mode	User mode
Privileges	Full hardware access	Restricted
Memory	Kernel space	User space
Examples	init, kthreadd, systemd	Word processor, browser
Crashes	Can crash system	Isolated

System processes provide essential OS services; user processes run applications.

TL;DR: System process = kernel mode, full access. User process = user mode, restricted.

Keyword/Key mappings: Kernel mode, user mode, privileges, process isolation, system

Q77: What is the Difference Between fork() and exec()?

Answer: fork() creates a new child process (copy of parent). exec() replaces the current process image with a new program.

Polished Answer: In Unix/Linux:

fork():

- Creates child process (duplicate of parent)
- Returns child PID to parent, 0 to child
- Uses Copy-on-Write for efficiency
- Parent and child continue from same point

exec():

- Replaces current process with new program
- Never returns on success
- Does not create new process (transforms existing)
- Preserves PID, file descriptors

Typical pattern: fork() then exec() to run new program in child.

TL;DR: fork() = create copy of process. exec() = replace process with new program. Often used together.

Keyword/Key mappings: Process creation, Copy-on-Write, process image, Unix process model

Q78: What is a Shell?

Answer: A shell is a command-line interpreter providing an interface between users and the OS, interpreting and executing commands.

Polished Answer: Shell functions:

- Parse and execute commands
 - Support scripting (shell scripts)
 - Provide programming constructs (loops, conditions)
 - Manage processes (background, redirection, pipes)
-

- Environment variable management

Types: Bourne shell (sh), Bash, Zsh, Fish, PowerShell.

TL;DR: Shell = command-line interface to OS. Interprets commands, supports scripting.

Keyword/Key mappings: Command interpreter, shell scripting, process management, environment variables

Q79: What is a Daemon Process?

Answer: A daemon is a background process that runs continuously, providing services without direct user interaction.

Polished Answer: Daemon characteristics:

- Runs in background (detached from terminal)
- Starts at boot or on-demand
- No controlling terminal
- Often runs with special permissions
- Logs to files, not console

Examples: httpd (web server), sshd (SSH server), cron (scheduler), syslogd (logging).

TL;DR: Daemon = background service process. No terminal, runs continuously.

Keyword/Key mappings: Background process, system service, init/systemd, detached process

Q80: What is a Signal in OS?

Answer: A signal is a software notification sent to a process to indicate an event requiring attention.

Polished Answer: Signals are async notifications:

Common signals:

- SIGINT (Ctrl+C): Interrupt
- SIGKILL: Force kill (can't be caught)
- SIGTERM: Terminate (can be handled)
- SIGSEGV: Segmentation fault
- SIGCHLD: Child process terminated

Handling: Process can catch signal, ignore it, or take default action.

TL;DR: Signal = software interrupt notifying process of events. Can be caught or ignored (except SIGKILL).

Keyword/Key mappings: Interrupt, process termination, signal handler, async notification

Q81: What is a File Descriptor?

Answer: A file descriptor is an integer identifying an open file or I/O resource within a process.

Polished Answer: File descriptors are per-process references to open files:

Standard file descriptors:

- 0: Standard input (stdin)
- 1: Standard output (stdout)
- 2: Standard error (stderr)

Usage: read(fd, buffer, size), write(fd, data, size). Each process has a file descriptor table mapping FDs to open file entries.

TL;DR: File descriptor = integer handle to open file/I/O. 0=stdin, 1=stdout, 2=stderr.

Keyword/Key mappings: Open file table, read/write system calls, file I/O, process resources

Q82: What is the Difference Between Port and Socket?

Answer: A port is a logical endpoint on a machine identified by a number. A socket is the combination of IP address + port, representing a communication endpoint.

Polished Answer:

Aspect	Port	Socket
Definition	Logical endpoint number	IP:port combination
Identification	Number (0-65535)	Address + port
Purpose	Identify service/process	Communication endpoint
Example	Port 80 (HTTP)	192.168.1.5:80

Server listens on a port; clients connect to server's socket.

TL;DR: Port = number identifying service. Socket = IP + port, actual communication endpoint.

Keyword/Key mappings: Network programming, TCP/IP, connection endpoint, port numbers

Q83: What is a Thread-Local Storage (TLS)?

Answer: Thread-Local Storage provides each thread with its own private copy of variables, avoiding synchronization for thread-specific data.

Polished Answer: TLS allows each thread to have separate variable instances:

Benefits:

- No synchronization needed for thread-specific data
- Safe access to global-like variables
- Better performance than shared variables with locks

Use cases: Error codes (errno), transaction contexts, session data.

Implementation: Compiler-level (`__thread` in C/C++) or OS-level APIs.

TL;DR: TLS = per-thread variable copies. No synchronization needed for thread-specific data.

Keyword/Key mappings: Thread isolation, synchronization avoidance, errno, compiler directives

Q84: What is the Difference Between Static and Dynamic Linking?

Answer: Static linking copies all required library code into the executable at compile time. Dynamic linking resolves library references at runtime using shared libraries.

Polished Answer:

Aspect	Static Linking	Dynamic Linking
Executable size	Larger	Smaller
Library updates	Requires recompile	Automatic (load new .so)
Startup time	Faster (no lookup)	Slower (resolve symbols)
Memory	Each process has copy	Shared among processes
Deployment	Self-contained	Requires libraries present

Modern systems prefer dynamic linking for flexibility and memory efficiency.

TL;DR: Static = libraries in executable. Dynamic = shared libraries at runtime. Dynamic saves memory.

Keyword/Key mappings: Shared libraries (.so/.dll), executable size, symbol resolution, deployment

Q85: What is a Core Dump?

Answer: A core dump is a file containing a process's memory image when it crashes, used for debugging.

Polished Answer: Core dumps capture process state at crash:

- Memory contents
- Register values
- Call stack
- Program counter

Usage: Debug with gdb: `gdb program core`. Analyze crash cause.

Control: `ulimit -c` to enable/disable. Windows equivalent: crash dumps.

TL;DR: Core dump = memory snapshot on crash for debugging. Analyze with debugger.

Keyword/Key mappings: Process crash, debugging, memory image, gdb, fault analysis

Q86: What is a Watchdog Timer?

Answer: A watchdog timer resets the system if it's not periodically refreshed, detecting system hangs or failures.

Polished Answer: Watchdog operation:

1. Timer set to expire after N seconds
2. Software periodically resets timer (health check)
3. If timer expires (system hung), hardware reset triggered

Use cases: Embedded systems, high-availability servers, mission-critical systems.

TL;DR: Watchdog = timer requiring periodic reset. System reset if not refreshed.

Keyword/Key mappings: System reliability, fault detection, embedded systems, hardware reset

Q87: What is a Hypervisor?

Answer: A hypervisor (VMM) is software creating and managing virtual machines, allowing multiple OS instances on one physical machine.

Polished Answer: Hypervisor types:

- **Type 1 (Bare-metal):** Runs directly on hardware (VMware ESXi, Xen, KVM)
- **Type 2 (Hosted):** Runs on host OS (VirtualBox, VMware Workstation)

Functions:

- Virtualize CPU, memory, I/O
- Isolate VMs from each other
- Allocate physical resources to VMs

TL;DR: Hypervisor = VM manager. Type 1 (bare-metal) or Type 2 (hosted).

Keyword/Key mappings: Virtualization, VMM, resource isolation, bare-metal, virtual machines

Q88: What is Containerization vs Virtualization?

Answer: Containers share host OS kernel; VMs have separate OS. Containers are lighter, VMs provide stronger isolation.

Polished Answer:

Aspect	Containers	Virtualization
OS	Shared host kernel	Each VM has own OS
Size	MB (lightweight)	GB (full OS)
Startup	Seconds	Minutes
Isolation	Process-level	Hardware-level
Performance	Near-native	Slight overhead
Example	Docker, Kubernetes	VMware, KVM

Containers for microservices; VMs for strong isolation.

TL;DR: Containers = share kernel, lightweight. VMs = full OS each, stronger isolation.

Keyword/Key mappings: Docker, kernel sharing, resource isolation, microservices, cloud computing

Q89: What is a Microservices Architecture?

Answer: Microservices decompose an application into small, independent services that communicate over the network.

Polished Answer: Microservices characteristics:

- Each service is independently deployable
- Services communicate via APIs (HTTP, gRPC)
- Polyglot (different tech stacks per service)
- Independent scaling
- Decentralized data management

vs Monolith: Monolith = one large application. Microservices = many small services.

TL;DR: Microservices = small independent services communicating via APIs. Scalable but complex.

Keyword/Key mappings: Service decomposition, API communication, independent deployment, scalability

Q90: What is Load Balancing?

Answer: Load balancing distributes workload across multiple computing resources to optimize resource use and prevent overload.

Polished Answer: Load balancing algorithms:

- Round Robin: Sequential distribution
- Least Connections: Send to least busy server
- IP Hash: Same client to same server
- Weighted: Based on capacity

Benefits: High availability, scalability, fault tolerance.

Implementation: Hardware load balancers or software (Nginx, HAProxy).

TL;DR: Load balancing = distribute work across servers. Algorithms: round robin, least connections, etc.

Keyword/Key mappings: High availability, server distribution, failover, throughput optimization

Q91: What is a Message Queue?

Answer: A message queue is a communication mechanism enabling asynchronous message passing between components or services.

Polished Answer: Message queues provide:

- **Asynchronous communication:** Sender doesn't wait
- **Decoupling:** Services work independently
- **Buffering:** Handle traffic spikes
- **Reliability:** Message persistence

Examples: RabbitMQ, Kafka, AWS SQS.

Patterns: Publish-subscribe, point-to-point, request-reply.

TL;DR: Message queue = async message passing between services. Enables decoupling and buffering.

Keyword/Key mappings: Async communication, publish-subscribe, RabbitMQ, Kafka, service decoupling

Q92: What is a Transaction?

Answer: A transaction is a sequence of operations treated as a single atomic unit, ensuring consistency.

Polished Answer: ACID properties:

- **Atomicity:** All or nothing
- **Consistency:** Valid state transitions
- **Isolation:** Concurrent transactions don't interfere
- **Durability:** Committed changes persist

Use: Databases, file systems, distributed systems.

TL;DR: Transaction = atomic operation group. ACID: Atomic, Consistent, Isolated, Durable.

Keyword/Key mappings: ACID, atomicity, rollback, commit, isolation levels

Q93: What is the Difference Between Synchronous and Asynchronous I/O?

Answer: Synchronous I/O blocks the process until operation completes. Asynchronous I/O allows the process to continue while I/O happens in the background.

Polished Answer:

Aspect	Synchronous I/O	Asynchronous I/O
Blocking	Process waits	Process continues
Complexity	Simple	More complex
Efficiency	Lower	Higher
Implementation	Regular read/write	epoll, io_uring, completion callbacks

Async I/O enables high-performance servers handling many connections (Nginx, Node.js).

TL;DR: Sync I/O = block until done. Async I/O = continue while I/O happens. Async scales better.

Keyword/Key mappings: epoll, non-blocking I/O, event-driven, high-performance servers, callback

Q94: What is a State Machine in OS Context?

Answer: A state machine models system states and transitions, used for process states, scheduling, and protocol handling.

Polished Answer: OS uses state machines for:

- Process states (New Ready Running Waiting Terminated)
- File system states
- Device driver states
- Network protocol states

Components: States, transitions, events triggering transitions.

TL;DR: State machine = model of states and transitions. Used for process lifecycle, protocols.

Keyword/Key mappings: Process states, transitions, events, modeling, system behavior

Q95: What is the Difference Between Hard Link and Soft Link?

Answer: A hard link points to the same inode as the original file. A soft link (symlink) contains the path to another file.

Polished Answer:

Aspect	Hard Link	Soft Link
Inode	Same inode	Different inode
Cross-filesystem	Not possible	Possible
Directories	Generally not allowed	Allowed
Original deletion	Link still works	Link breaks
Example	In file link	In -s file link

TL;DR: Hard link = same inode, survives original deletion. Soft link = path pointer, breaks if target removed.

Keyword/Key mappings: Inode sharing, symbolic link, file system, link count

Q96: What is Journaling in File Systems?

Answer: Journaling records intended changes to a journal before applying them, enabling recovery after crashes.

Polished Answer: Journaling (write-ahead logging) prevents corruption:

1. Write intended changes to journal
2. Apply changes to file system
3. Mark journal entry complete
4. On crash, replay/rollback incomplete operations

Benefits: Fast recovery after power failure, data consistency.

Examples: ext4, NTFS, XFS, APFS.

TL;DR: Journaling = log changes before applying. Faster crash recovery, prevents corruption.

Keyword/Key mappings: Write-ahead logging, crash recovery, ext4, NTFS, data consistency

Q97: What is a Buffer Overflow?

Answer: A buffer overflow occurs when data exceeds a buffer's allocated size, overwriting adjacent memory.

Polished Answer: Buffer overflow consequences:

- Data corruption
- Program crashes
- Security vulnerabilities (arbitrary code execution)

Prevention:

- Bounds checking (safe functions: strncpy vs strcpy)
- ASLR (Address Space Layout Randomization)
- Stack canaries
- Non-executable stack
- Memory-safe languages (Rust, Java)

TL;DR: Buffer overflow = writing beyond buffer bounds. Security risk, prevented by bounds checking.

Keyword/Key mappings: Stack smashing, ASLR, stack canary, memory safety, bounds checking

Q98: What is a Memory Leak?

Answer: A memory leak occurs when allocated memory is not freed, causing the process to consume increasing memory over time.

Polished Answer: Memory leaks cause:

- Growing memory consumption
- Performance degradation
- Eventually system instability

Detection: Valgrind, AddressSanitizer, heap profilers.

Prevention: Smart pointers (C++), garbage collection (Java, Python), careful malloc/free management.

TL;DR: Memory leak = allocated memory never freed. Process memory grows unbounded.

Keyword/Key mappings: malloc/free, garbage collection, smart pointers, heap management, Valgrind

Q99: What is a Denial-of-Service (DoS) Attack?

Answer: A DoS attack overwhelms a system's resources, making it unavailable to legitimate users.

Polished Answer: DoS attack types:

- **Resource exhaustion:** CPU, memory, disk, network bandwidth
- **Application-level:** Exploiting vulnerabilities
- **Distributed (DDoS):** From multiple sources

OS-level defenses: Rate limiting, resource quotas, input validation, connection limits.

TL;DR: DoS = overwhelm system resources. Defense: rate limiting, quotas, validation.

Keyword/Key mappings: Resource exhaustion, DDoS, rate limiting, system availability, security

Q100: What is a File Lock?

Answer: A file lock prevents concurrent processes from modifying the same file simultaneously, ensuring data consistency.

Polished Answer: File lock types:

- **Advisory locks:** Processes voluntarily respect (flock, lockf)
- **Mandatory locks:** Enforced by OS
- **Shared locks:** Multiple readers allowed
- **Exclusive locks:** One writer only

Use: Database files, log files, shared configuration.

TL;DR: File lock = prevent concurrent file modification. Advisory or mandatory, shared or exclusive.

Keyword/Key mappings: flock, file consistency, concurrent access, read/write locks, data integrity

SUMMARY TABLE

Rank Range	Focus Area
Top 10	Process/Thread, Deadlock, Virtual Memory, Scheduling, Kernel
11-25	Synchronization, IPC, Fragmentation, Page Replacement, System Calls
26-50	OS Architecture, Memory Management, File Systems, Advanced Sync
51-100	Specialized topics, Networking, Security, Modern Computing

Preparation Tips:

1. Master Top 10 completely (most frequently asked)
 2. Understand concepts, not just definitions
 3. Practice with real scenarios and examples
-

4. Know trade-offs and design decisions
5. Link concepts (memory scheduling synchronization)